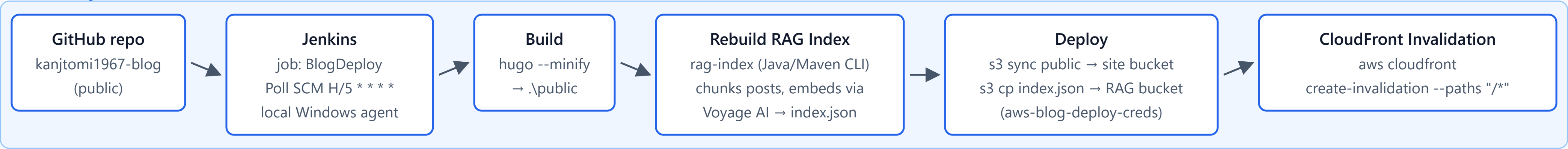


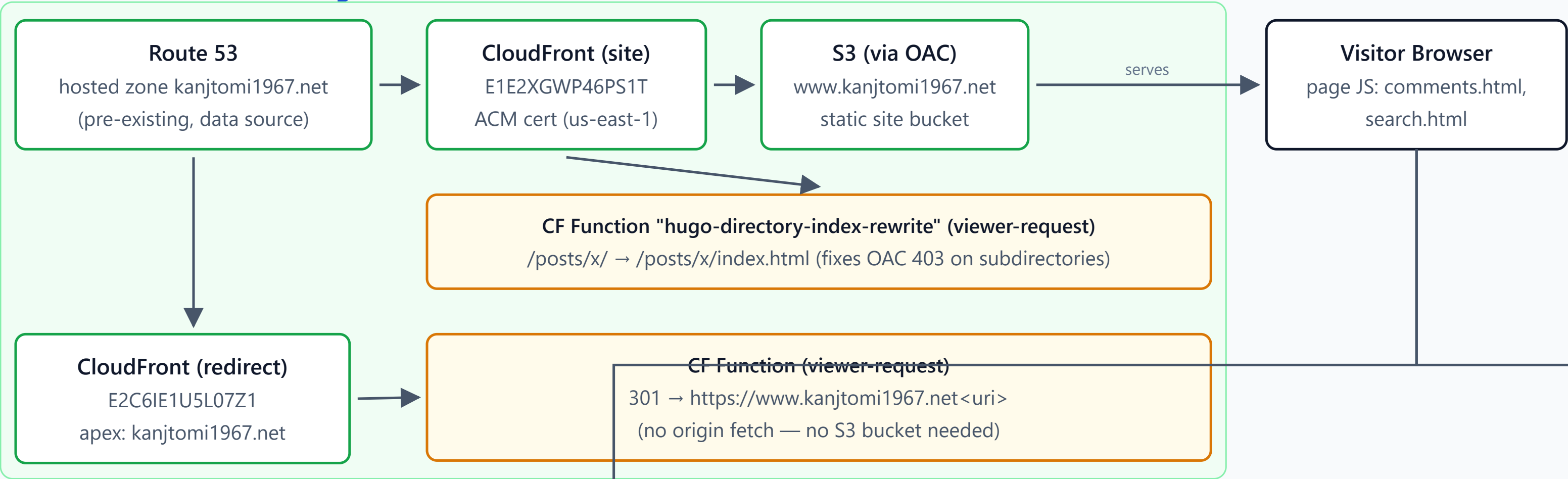
www.kanjtomi1967.net — Blog Architecture

Static Hugo site on AWS (S3 + CloudFront), self-hosted comments and RAG search via Lambda

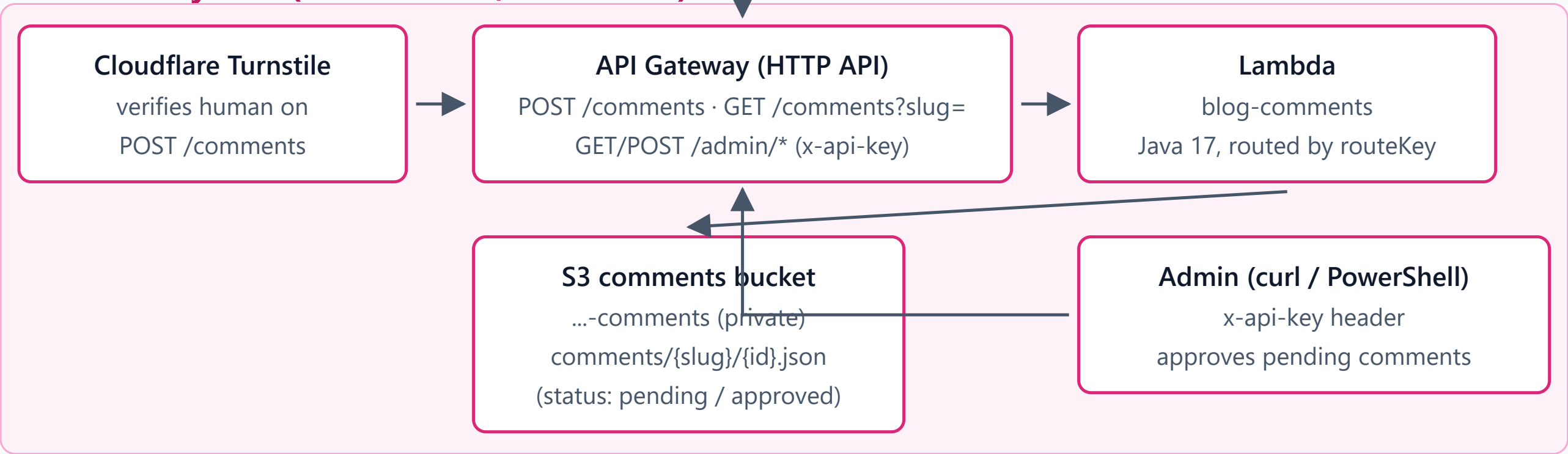
CI/CD Pipeline (local Jenkins on Windows)



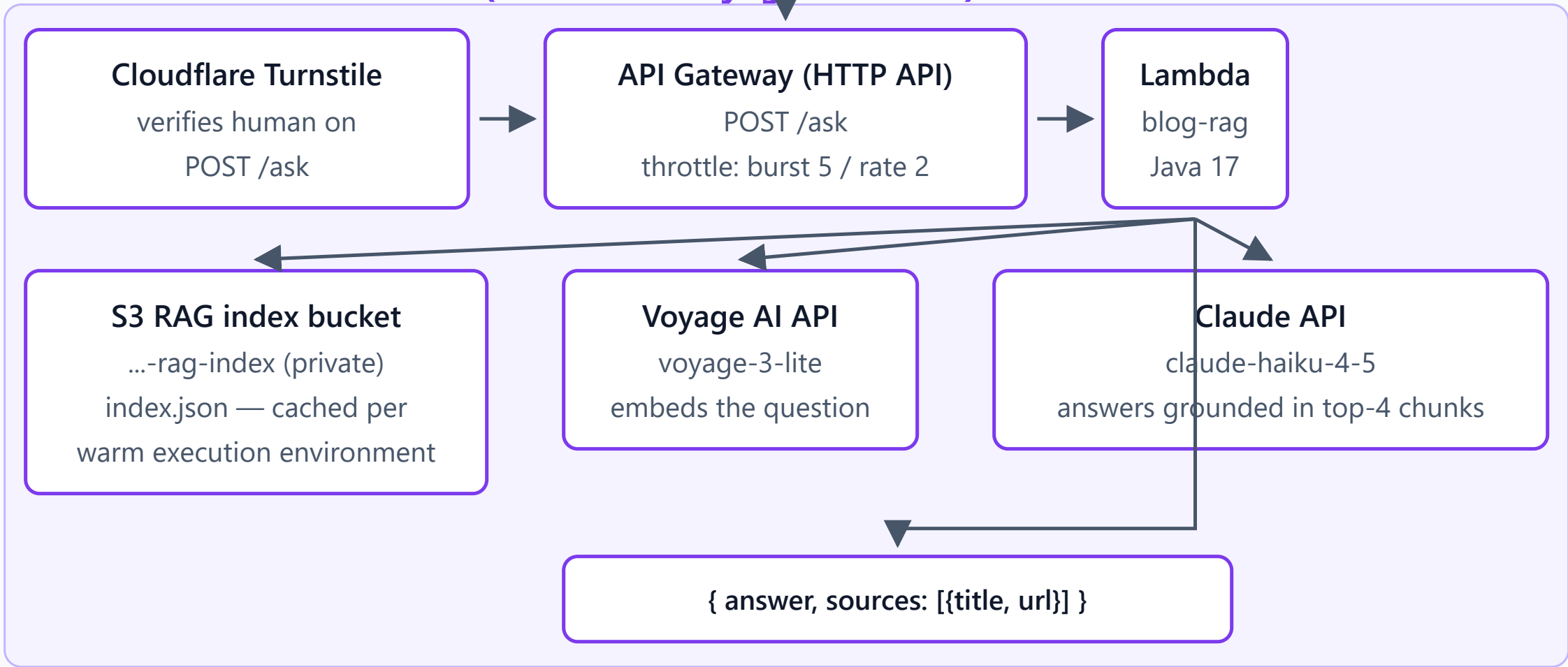
DNS, CDN & Static Hosting



Comment System (Lambda + S3, moderated)



RAG Search — "AI に聞く" (Lambda + Voyage + Claude)



Notes

Estimated cost: ~\$1-3/month (Route 53 zone + small redirect distribution); Lambda/API Gateway usage stays within/near free tier for personal traffic.
Secrets (Turnstile secret, admin_api_key, Voyage/Anthropic API keys) live only in terraform.tfvars (gitignored) and Lambda env vars — never committed.
Out of scope: no user login/accounts, no likes/reactions. Comments are anonymous + name field, moderated via x-api-key admin routes.
Region: ap-northeast-1 for all resources except ACM certificates, which must be requested in us-east-1 for CloudFront.

モジュール解説

CI/CD パイプライン(ローカル Jenkins on Windows)

GitHub repo

ソースを管理する公開リポジトリ(kanjtomi1967-blog)。Markdown記事、Hugo設定、Terraform、Lambdaコードなど、サイトを構成する全てをここで管理する。

Jenkins

ローカルWindows上で稼働する自前のCI/CDサーバー(ジョブ名 BlogDeploy)。GitHubから直接Webhookを受けられないため、5分間隔のPoll SCM(H/5 * * * *)で新しいコミットを検知してビルドを起動する。

Build

hugo --minify を実行し、Markdown記事を静的HTMLへビルドして .¥public に出力するステージ。

Rebuild RAG Index

rag-index(Java/Maven製CLI)が全記事を読み込み、見出し単位でチャンク分割してVoyage AIで埋め込みベクトル化し、index.jsonを生成する。デプロイのたびに実行され、常に最新記事を検索対象に反映する。

Deploy

ビルド済みサイトをS3の本番バケットへ `aws s3 sync`、RAGインデックスを専用S3バケットへ `aws s3 cp` でアップロードするステージ。Jenkins Credentials(aws-blog-deploy-creds)のIAM権限を使用する。

CloudFront Invalidation

デプロイ内容を即座に反映させるため、`aws cloudfront create-invalidation --paths "/*"` でCDNキャッシュを全パス無効化する。

DNS・CDN・静的ホスティング

Route 53

ドメイン kanjtomi1967.net の既存ホストゾーン。Terraformではデータソースとして参照するのみで、ゾーン自体は管理対象外(既存インフラ)。

CloudFront(site)

www.kanjtomi1967.net 向けの本番配信ディストリビューション。オリジンはS3(OAC経由)。CloudFront向けの制約により、TLS証明書は us-east-1 で発行したACM証明書を使用する。

S3(via OAC)

ビルド済み静的サイトを格納する非公開バケット。パブリックアクセスは一切許可せず、CloudFrontのOrigin Access Control経由でのみ読み取り可能。

CF Function "hugo-directory-index-rewrite"

/posts/x/ のようなディレクトリ形式のURIを /posts/x/index.html へ、viewer-request時に書き換えるエッジ関数。OAC経由のS3はルート以外のディレクトリindexを自動解決しないため、これがないと403 AccessDeniedになる。

CloudFront(redirect)

apexドメイン(kanjtomi1967.net)専用の、トラフィックがほぼ発生しない軽量の2つ目のディストリビューション。

CF Function(apex redirect)

全リクエストを https://www.kanjtomi1967.net<uri> へ301リダイレクトするエッジ関数。オリジンへのフェッチが一切発生しないため、apex用の公開S3バケットが不要(アカウントのS3 Block Public Accessとも衝突しない)。

Visitor Browser

実際にサイトを閲覧するユーザーのブラウザ。コメント欄・RAG検索のJS(comments.html / search.html)を実行し、それぞれのAPIを呼び出す。

コメントシステム(Lambda + S3、承認制)

Cloudflare Turnstile

コメント投稿(POST /comments)がbotではなく人間によるものかをサーバー側で検証する、CAPTCHA代替サービス。

API Gateway(HTTP API)

コメント関連の全エンドポイントをまとめるルーター。公開ルート(POST /comments、GET /comments?slug=)と、x-api-key必須の管理者ルート(/admin/*)を持つ。

Lambda blog-comments

Java 17製の単一Lambda。API GatewayのrouteKeyで内部的にルーティングし、投稿・取得・承認処理をすべて1つの関数で担う。

S3 comments bucket

非公開バケット(...-comments)。コメント1件ごとに comments/{slug}/{id}.json として保存し、pending / approved のstatusフィールドで管理する。

Admin(curl / PowerShell)

専用の管理UIは持たず、x-api-keyヘッダー付きのリクエストでpending中のコメントを一覧・承認する運用(scripts/comments-admin.md参照)。

RAG検索 — 「AI に聞く」 (Lambda + Voyage + Claude)

Cloudflare Turnstile

質問送信(POST /ask)側でも同様にbot対策として検証を行う。コメント側とは別実装だが同じ仕組みを再利用している。

API Gateway(HTTP API)

POST /ask の単一ルート。Claude API呼び出しコストが暴走しないよう、バースト5・レート2の保守的なスロットリングを設定している。

Lambda blog-rag

Java 17製。コールドスタート時にS3からindex.jsonをメモリに読み込み、ウォーム実行環境間で使い回す。質問文をベクトル化し、コサイン類似度で総当たりランキングして上位4チャンクを取得し、Claudeに渡す(ベクトルDBは使わず、この規模ではブルートフォースで十分という判断)。

S3 RAG index bucket

非公開バケット(...-rag-index)。rag-index CLIが生成した index.json(記事チャンクとその埋め込みベクトル式)を保管する。

Voyage AI API

voyage-3-liteモデルで、ユーザーの質問文を埋め込みベクトルに変換する。

Claude API

claude-haiku-4-5。取得した上位4チャンクのみを根拠として回答を生成する。ブログの内容でカバーされていない質問には、外部知識で答えず正直にその旨を伝えるよう指示されている。

レスポンス

{ answer, sources: [{title, url}] } 形式でフロントエンドに返却される。

備考

- 想定コスト: 月額 約\$1〜3(Route 53ホストゾーン + 小規模なりダイレクト用CloudFront)。CloudFront/S3/Lambda/API Gatewayの利用量は、個人ブログ規模のトラフィックであれば無料利用枠内〜ほぼ無料枠に収まる想定。
- シークレット管理: Turnstileのシークレットキー、admin_api_key、Voyage/Anthropic APIキーは terraform.tfvars(gitignore対象)とLambdaの環境変数にのみ存在し、リポジトリにはコミットされない。
- スcope外: ユーザーログイン/アカウント機能、いいね/リアクション機能は実装しない。コメントは匿名+名前欄のみで、承認制で運用する。
- リージョン: ACM証明書(CloudFront向け)のみ us-east-1、それ以外の全リソースは ap-northeast-1。